Module 3

Discrete-Time Dynamic Programming and Reinforcement Learning

Discrete-time dynamic programming of Bellman [1]-[2] is very closely related to reinforcement learning (RL). It represents the mathematical foundation and mathematical justification for RL techniques. The cases for which the optimal feedback solution can be found analytically are presented. These are the cases of the linear-quadratic (LQ) optimal control problem and the optimal control of affine nonlinear systems, which have numerous applications for real dynamic engineering systems. We present the Hewer algorithm [3]-[4] (analog to the continuous-time Kleinman algorithm [5]) that in fact represents the RL policy iteration solution to the discrete-time LQ optimal control problem. At the end of the module, we formulate a MATLAB RL design project based on Hewer's algorithm, in which a LQ reinforcer is designed by performing policy iterations using solutions of a set of discrete-time algebraic Lyapunov equation.

The original work of Bellman [1]-[2] was further developed by Bellman and his coworkers [6]-[7], and many other researchers in the years to come, most notable by Larson [8]-[9]. A comprehensive review of discrete-time dynamic programming can be found in [8], and in the books by Larson [9], Larson and Casti [10].

Due to computational difficulties, the original dynamic programming was not too much in use until the discovery of the *approximate dynamic programming* (ADP) at the beginning of this century. After the discovery of ADP in continuous-time by Murray *et al.* in 2002 [11], the most noticeable articles and books on discrete-time ADP are [12]-[21]. Moreover, revitalization of the dynamic programming in general, was significantly supported by researchers in computer science working on RL at the turn of the century. To honor Bellman, they named one of the most important equations of reinforcement learning, the *Bellman equation*.

The Bellman Functional Difference Equation

The dynamic optimization problem is formulated as follows. Consider a time-invariant nonlinear discrete-time dynamic system defined by

$$x(k+1) = f(x(k), u(k)), \quad x(k_0) = x_0, \quad k = k_0, k_0+1, k_0+2, \ldots \tag{3.1}$$

where $x(k) \in R^n$ is the system state space vector, $u(k) \in R^m$ is the system control input vector, $x(k_0)$ is the system initial state, and k stands for discrete time with k_0 is the initial time. A scalar performance criterion to be optimized, associated to this dynamic system, is defined by

$$J(x_{k_0}, k_0) = \sum_{k=k_0}^{k_f-1} g(x(k), u(k)) + \gamma(x(k_f)) \tag{3.2}$$

where $g(x(k), u(k))$ is a *nonnegative scalar function*, representing the value of the performance criterion (cost) at the discrete time k, k_f is the final time, and $\gamma(x(k_f)) \geq 0$ is the final time constraint imposed on the state variables. The goal is to find the optimal value of (3.2) along trajectories of dynamic system (3.1). Since $\gamma(x(k_f))$ is known, optimization can be done backwards in time by performing multi-stage *static* optimization starting at the terminal time $k = k_f$ with $J_{k_f}^{\text{opt}}(x(k_f)) = \gamma(x(k_f))$, and going backwards in time $k = k_f - 1,\ k_f - 2, ..., k_0$, using (3.1) and (3.2). This produces a sequence of static optimization problems whose solutions lead to

$$\begin{aligned} &J_j^{\text{opt}}(x(j), j) = \min_{u(j)} \left\{ g(x(j), u(j) + J_{j+1}^{\text{opt}}(f(x(j), u(j)) \right\} \quad \Rightarrow \quad u(j) = u(x(j)) \\ &j = k_f - k_0, k_f - k_0 - 1, ..., 2, 1, 0, \end{aligned} \tag{3.3}$$

Equation (3.3) represents the fundamental equation of dynamic programming known as the *Bellman functional equation of dynamic programming in discrete-time*. The quantity in the large bracket defines the discrete-time *Hamiltonian*. It is denoted by $H(x(j), u(j))$ and defined by

$$H(x(j), u(j)) = g(x(j), u(j)) + J_{j+1}^{\text{opt}}\left(f(x(j), u(j)) \right) \tag{3.4}$$

where $g(x(j), u(j))$ is the instantaneous cost value at discrete-time $k = j$, and $J_{j+1}^{\text{opt}}(f(x(j), u(j)))$ represents the optimal cost to go from $j+1$ until the end of the optimization period.

The *necessary condition for minimum* in (3.3) at any discrete-time k implies

$$\frac{\partial}{\partial u} \left\{ g(x(k), u(k)) + J_{k+1}^{\text{opt}}\left(f(x(k), u(k)) \right) \right\} = 0 \quad \Rightarrow \quad u(x(k)) \tag{3.5}$$

indicating that the functions $g(x(k), u(k))$ and $f(x(k), u(k))$ must be differentiable with respect to control variable $u(k)$. If the optimal feedback control exists, the optimal performance is

$$J^{\text{opt}}\left(x_{k_0}, k_0 \right) = \sum_{k=k_0}^{k_f-1} g(x^{\text{opt}}(k), u^{\text{opt}}(x^{\text{opt}}(k)) + \gamma(x^{\text{opt}}(k_f)) \tag{3.6}$$

Comment 3.1: *Note that the obtained algebraic equation (3.5) must have a unique solution in each optimization stage.* Since for RL a feedback solution is required, it follows that the optimal feedback control can be found in a very limited number of cases. Fortunately, for linear systems with quadratic performance criteria (LQ optimization) a unique solution exists under a very mild condition at each discrete time instant.

Linear-Quadratic Optimal Control Problem in Discrete Time

Consider a linear discrete-time system with a quadratic performance criterion

$$x(k+1) = Ax(k) + Bu(k), \quad x(0) = x_0 \tag{3.7}$$

$$J = \frac{1}{2}\sum_{k=0}^{N-1}\left[x^T(k)R_1x(k) + u^T(k)R_2u(k)\right] + \frac{1}{2}x^T(N)R_{1N}x(N) \tag{3.8}$$

$$R_1 = R_1^{\mathrm{T}} \geq 0,\ R_2 = R_2^{\mathrm{T}} > 0,\ R_{1N} = R_{1N}^{\mathrm{T}} \geq 0$$

Note that in the performance criterion (3.8), the used notation (compared to the original definition in (3.2)) is slightly simplified by taking $k_0 = 0$ and $k_f = N$. The solution to the linear-quadratic (LQ) optimization problem can be obtained using multi-stage optimization, which in fact is a backward in time recursive procedure. To that end, the corresponding *dynamic optimization problem is replaced by a sequence of the static optimization problems* at each discrete-time instant.

The following expressions are obtained in the stage $k = N - K$

$$\begin{aligned} u^{\mathrm{opt}}(N-K) &= -\left[R_2 + B^{\mathrm{T}}P(K-1)B\right]^{-1} B^{\mathrm{T}}P(K-1)Ax(N-K) \\ &= -F^{\mathrm{opt}}(N-K)x(N-K) \end{aligned} \tag{3.9}$$

$$\begin{aligned} J_{N-K,N}^{\mathrm{opt}} = \frac{1}{2}x^{\mathrm{T}}(N-K)\Big\{&\left[A - BF(N-K)\right]^{\mathrm{T}} P(K-1)\left[A - BF(N-K)\right] \\ &+ F^{\mathrm{T}}(N-K)R_2F(N-K) + R_1\Big\}x(N-K) = \frac{1}{2}x^{\mathrm{T}}(N-K)P(K)x(N-K) \end{aligned} \tag{3.10}$$

$$\begin{aligned} P(K) = &\left[A - BF(N-K)\right]^{\mathrm{T}} P(K-1)\left[A - BF(N-K)\right] \\ &+ F^{\mathrm{T}}(N-K)R_2F(N-K) + R_1, \qquad P(0) = R_{1N}, \quad K = 1,2,\ldots,N \end{aligned} \tag{3.11}$$

In the last stage of the *N*-stage optimization process, $k = N - N = 0$, the optimal performance becomes

$$J_{0,N}^{\mathrm{opt}}(x_0) = \frac{1}{2}x^{\mathrm{T}}(0)P(N)x(0) \tag{3.12}$$

Note that at the stage $k = N - K$, the original time invariant system (3.1), after replacement of optimal control (3.9) becomes a time varying system, that is, its feedback matrix $A + BF(N-K)$ is time varying. When $N \to \infty$ (optimization over an infinite discrete time period, steady state optimization) and with $R_{1N} = 0$, under controllability (stabilizability) and observability (detectability) conditions, the matrices F and P tend to constant matrices, and we are faced with problems of solving the algebraic equations given by

$$P = \left(A - BF^{\text{opt}}\right)^{\text{T}} P\left(A - BF^{\text{opt}}\right) + F^{\text{opt}^{\text{T}}} R_2 F^{\text{opt}} + R_1 \tag{3.13}$$

$$F^{\text{opt}} = \left(R_2 + B^{\text{T}} PB\right)^{-1} B^{\text{T}} PA \tag{3.14}$$

After some algebra, the optimal gain F^{opt} can be eliminated from equation (3.13), and we get the well-known *discrete-time Riccati algebraic equation (DARE)* defined by

$$P = A^{\text{T}} PA + R_1 - A^{\text{T}} PB\left(R_2 + B^{\text{T}} PB\right)^{-1} B^{\text{T}} PA \tag{3.15}$$

Under control systems oriented assumptions, controllability (stabilizability) and observability (detectability) DARE has a unique positive semi-definite solution that stabilizes the optimal feedback system

$$x^{\text{opt}}(k+1) = \left(A - BF^{\text{opt}}\right) x^{\text{opt}}(k), \quad x(0) = x_0 \tag{3.16}$$

that is, the feedback matrix $A - BF^{\text{opt}}$ is asymptotically stable (it has all its eigenvalues strictly inside of the unit circle, that is, the following is satisfied: $\left|\lambda_i (A - BF^{opt})\right| < 1, \forall i$).

Assumption 3.1: *The discrete-time algebraic Riccati equation (3.15) has a unique solution that stabilizes the closed-loop system under the assumption that the pair* (A, B) *is stabilizable the pair* $(A, \text{Chol}(R_1))$ *is detectable.*

Stabilizability and *detectability* are weaker assumptions of respectively well-known controllability and observability assumptions. Namely, they require that only unstable eigenvalues of the system matrix A are respectively controllable and observable. Homework problems 3.5 and 3.6 provide analytical conditions how stabilizability and detectability can be tested within the requirements that students write live functions for testing stabilizability and detectability. It is interesting to point out that the Control System Toolbox of MATLAB has no functions for examining system stabilizability and system detectability.

Calculus of Variations and Discrete-Time Hamiltonian

For the discrete-time, time-invariant, system (3.1) and the corresponding performance criterion (3.2), that is

$$x(k+1) = f(x(k), u(k)), \quad x(k_0) = x_{k_0}, \quad k = k_0, k_0 + 1, k_0 + 2, \ldots$$

$$J\left(x_0, k_0\right) = \sum_{k=k_0}^{N-1} g(x(k), u(k)) + \gamma(x(N))$$

the mathematical dynamic optimization theory based on calculus of variations, defines the *discrete-time Hamiltonian* as

$$H(x(k), u(k), p(k+1)) = g(x(k), u(k)) + p^{\text{T}}(k+1) f(x(k), u(k)) \tag{3.17}$$

where $p(k)$ represents the so-called costate variable defined by

$$p(k)=\frac{\partial}{\partial x(k)}\{H(x(k),u(k),p(k+1))\},\qquad p(N)=\frac{\partial}{\partial x}\{\gamma(x(N))\} \tag{3.18}$$

The state equation can be obtained from the Hamiltonian by taking the partial derivative

$$x(k+1)=\frac{\partial H(x(k),u(k),p(k+1))}{\partial p(k+1)}=f(x(k),u(k)),\quad x(k_0)=x_{k_0} \tag{3.19}$$

According to the calculus of variations difference equations (3.22) and (3.23) together with the algebraic equation

$$0=\frac{\partial}{\partial u(k)}\{H(x(k),u(k),p(k+1),k)\}\quad\Rightarrow\quad u(x(k),p(k+1)) \tag{3.20}$$

constitute a set of three necessary conditions for the optimum in the discrete-time optimization problem defined by (3.1) and (3.2). The crucial step is that the analytical solution of the algebraic equation (3.20) for $u(x(k),p(k+1))$ exists. In such a case, $u(x(k),p(k+1))$ can be substituted in difference equations (3.22)-(3.23) to form the system of difference equations

$$\begin{aligned}&x(k+1)=f(x(k),u(x(k),p(k+1))),\quad x(k_0)=x_{k_0}\\&p(k)=\frac{\partial}{\partial x(k)}\{H(x(k),u(x(k),p(k+1)),p(k+1))\},\quad p(N)=\frac{\partial}{\partial x}\{\gamma(x(N))\}\end{aligned} \tag{3.21}$$

The system of difference equations (3.21) is known as the two-point boundary value problem (TPBVP) since for a part of unknowns, $x(k)$, the initial condition is known and the remaining variables, $p(k)$, have the terminal condition specified. It will be seen in this chapter and in the follow-up sections of this book that in some important cases, such as linear systems with quadratic performance criteria and affine nonlinear systems with quadratic performance criteria, the unique solutions to the corresponding optimal control problems exist.

In addition to the necessary conditions, the *sufficient condition for minimum* requires that the Hamiltonian's Hessian matrix is positive definite, that is

$$\frac{\partial^2 H(x(k),u(k),p(k+1))}{\partial u^2(k)}>0 \tag{3.22}$$

LQ Optimal RL Policy Iteration Design Project – Hewer's Algorithm

In this design project, students will need to use MATLAB and Simulink to demonstrate the RL mechanism of the LQ optimal control (reinforcer) in discrete-time, known as the Hewer algorithm [3]-[4]. This project is appropriate for senior undergraduate and first year graduate students. It is convenient to use the MATLAB live scripts with two sliders for the number of policy iterations and for the length of optimization time interval.

The project report is supposed to be typed. It must contain all standard technical paper main parts: introduction, problem formulation, main simulation results obtained, discussion and comments on obtained results and observations made, conclusions, appendix (with MATLAB codes), references. The report should contain a brief description of the Hewer policy iteration algorithm with analytical presentation of all important formulas, equations, required conditions (assumptions).

The project considers a Boeing 747 flight controls system from [23]. Assuming level flight at $40\,000\,\text{ft}$ at a speed of $774\,\text{ft/s}$, and the discretization interval of $1\,\text{s}$, the linearized aircraft dynamic is represented in the discrete-time domain by

$$x(k+1) = Ax(k) + Bu(k)$$

where

$$A = \begin{bmatrix} 0.99 & 0.03 & -0.02 & -0.32 \\ 0.01 & 0.47 & 4.7 & 0.0 \\ 0.02 & -0.06 & 0.4 & 0.0 \\ 0.01 & -0.04 & 0.72 & 0.99 \end{bmatrix}, \qquad B = \begin{bmatrix} 0.01 & 0.99 \\ -3.44 & 1.66 \\ -0.83 & 0.44 \\ -0.47 & 0.25 \end{bmatrix}$$

The penalty matrices in the corresponding LQ optimal control problem were selected in [23] as, $R_1 = I_4, R_2 = I_2$. Note that in MATLAB, the algebraic Lyapunov and Riccati equations in discrete-time can be solved respectively using the corresponding MATLAB functions `dlyap` and `dlqr`. The MATLAB commands: `>> help dlyap` and `>> help dlqr` provide all necessary information about the usage of these MATLAB functions.

Running policy iterations of the Hewer algorithm generate a table that shows how the approximate performance values converge per iteration to the optimal values. In general, you need to run only five to six iterations, in rare case nine or ten iterations due to the quadratic convergence of the algorithm, as established in [3]-[4]. Plot on one figure one of the approximates feedback controls obtained for $k = 1,2,3,4,5$, and plot on another figure the approximate system state trajectories for a selected (the one that performs the worst) state variable for $k = 1,2,3,4,5$.

Hint: Since matrix A is asymptotically stable, that is $\left|\lambda_j(A)\right| < 1$ for every $j = 1,2,...,n$, the initial feedback gain F_0 can be used as a zero matrix. Since we are dealing with vectors and matrices, the dimensions of all matrices are important. Hence, in this example, in MATLAB we input `>> F0=zeros(2,4)`.

Essence of the Hower RL Policy Iteration Algorithm

The essence of the Hewer algorithm [3]-[4] is to use successive approximations (feedback iterations $u_i(x(k)) = -F_i x(k), i = 1,2,...$, which is equivalent to the RL policy iterations) as follows

$$P_i = \left(A + BF_{i-1}\right)^{\mathrm{T}} P_i \left(A + BF_{i-1}\right) + F_{i-1}^{\mathrm{T}} R_2 F_{i-1} + R_1, \quad i = 1,2,3,...$$

$$F_i = -\left(R_2 + B^{\mathrm{T}} P_i B\right)^{-1} B^{\mathrm{T}} P_i A \quad \text{with } F_0 \text{ stabilizing } \left|\lambda_j(A - BF_0)\right| < 1$$

This requires solving in each iteration a discrete algebraic Lyapunov equation (DALE) and updating the approximate feedback gain after initializing the procedure with a stabilizing feedback gain F_0, which is much simple and faster than directly solving highly nonlinear algebraic discrete Riccati equation. These iterations converge to the unique stabilizing solution of DARE, [3]-[4].

$$P = A^{\mathrm{T}}PA + R_1 - A^{\mathrm{T}}PB\left(R_2 + B^{\mathrm{T}}PB\right)^{-1} B^{\mathrm{T}}PA$$

$$P_1 \geq P_2 \geq P_3 \geq \cdots \geq P$$

$$J^{\mathrm{opt}}(x_0) = \frac{1}{2} x_0^{\mathrm{T}} P x_0$$

MATLAB Hewer's policy iteration code for an inverted pendulum on a cart RL is presented.

MATLAB RL-Design-Project Demo 3.1:

```
% RL for an Inverted Pendulum on a Cart - DEMO
clear all
% Continuous-time inverted pendulum on a cart model
Ac=[0 0 1 0;
    0 0 0 1;
   0 40.2 -14.53 0;
   0 82.56 -15.77 0];
Bc=[0;0;27.28;26.74];
Cc=[1 0 0 0; 0 1 0 0];
Dc=[0;0];
sysc=ss(Ac,Bc,Cc,Dc); % continuous-time state space model
% this model can be discretized using MATLAB for the given
Ts=0.1 % selected the sampling period Ts
sysd=c2d(sysc,Ts); % discrete-time state space model
% extraction of discrete-time state space matrices
[Ad,Bd,Cd,Dd]=ssdata(sysd);
x0=[1;1;1;1]; % assumed initial conditions
% LQ Optimal Control
R1=0.01*eye(4); R2=1;
% assumed performance criterion penalty matrices
% Test for controllability and observability
CO=ctrb(Ad,Bd);
rankCO=rank(CO) % rank of the controllability matrix
% rank equal to n, system is controllable
OM=obsv(Ad,R1); % rank of the observability matrix
rankOM=rank(OM)
% rank equal to n, system is  observable
% the same rank is obtained by using obsv(Ad,Chol(R1))
% direct solution of the discrete optimal LQ problem
[Fopt,P]=dlqr(Ad,Bd,R1,R2);Jopt=0.5*x0'*P*x0
```

```
% Reinforcement learning solution
ev=eig(Ad);
% To find F0 that stabilize the system, we can use the
% MATLAB function "place" that finds the feedback gain
% matrix that places the closed-loop eigenvalues of
% Ad-Bd*F0 in the desired locations
lambda_des=[-0.5;0.2;-0.1+j*0.5;-0.1-j*0.5];
F0=place(Ad,Bd,lambda_des);
Fi=F0
% check eigenvalues Ad-Bd*F0; evF0=eig(Ad-Bd*F0);
% Hewer's Discrete-Time LQ Optimal Reinforcer
for i=1:15
    Pi=dlyap((Ad-Bd*Fi)',R1+Fi'*R2*Fi);
    i
    Ji(i)=0.5*x0'*Pi*x0
    Fi=inv(R2+Bd'*Pi*Bd)*Bd'*Pi*Ad;
    k=1:1:100;
    % evaluating approximate states and control
for j=1:100
     x(:,j)=((Ad-Bd*Fi)^k(j))*x0;
     u(j)=-Fi*x(:,j);
end
% plotting approximate states, control, performance
figure (1)
stairs(k,x(1,:)); hold on
xlabel('Time [s]');ylabel('State variable x1'); grid
figure (2)
stairs(k,u(k)); hold on
xlabel('Time [s]');ylabel('Approximate control'); grid
axis([0 20 -8 10])
end
figure (3)
i=1:1:15
plot(i,Ji,'*')
grid; xlabel('iteration number');
ylabel('approximate optimal performance')
% it is nicer to present the changes of Ji in a table which
% will also provide information about the decimal digits
% i=1,Ji=4269.3540;i=2,Ji=1080.0163;i=3,Ji= 442.5366;i=4,Ji= 237.4153;
% i=5,Ji=158.0270;i=6,Ji=123.0368;i=7,Ji=106.8144;i=8,Ji=99.3348;
% i=10,Ji=95.0129;i=13,Ji=94.8411=Jopt; i=15, Ji=94.8411
```

The obtained results are presented in Figures 3.1-3.3.

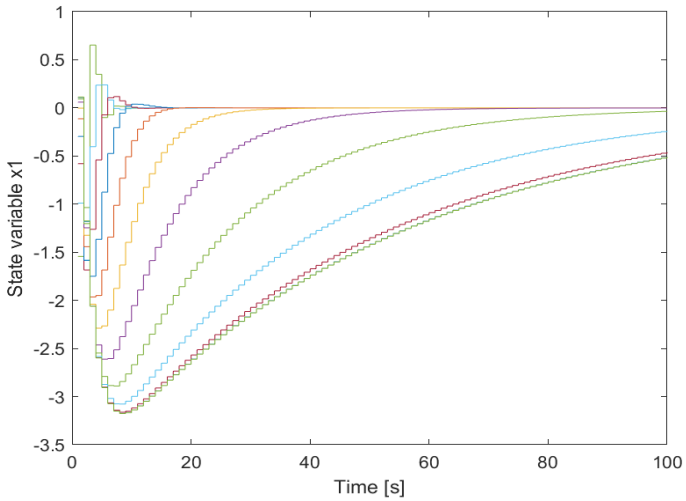


Figure 3.1: Approximate and optimal (the bottom one) state trajectories for $x_1(t)$

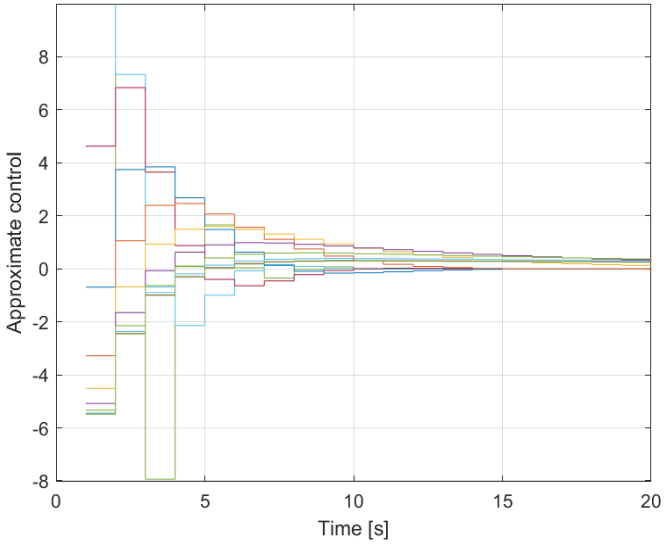


Figure 3.2: Approximate and optimal (the top one) control variables $u(t)$

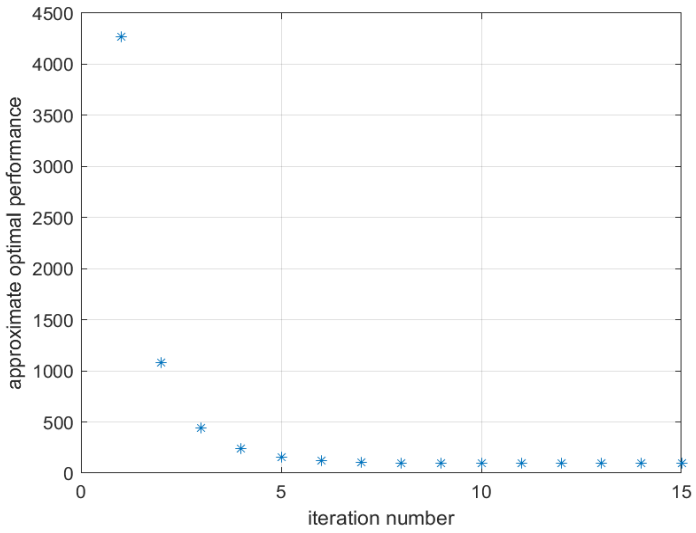


Figure 3.3: Approximate and optimal (steady state value) performance criteria

Using MATLAB live scripts, the Boeing 747 design project solution is presented in the next pages (the first two pages is the `.mlx` script, and the remaining pages are the results generated by the MATLAB live script).

MATLAB RL-Design-Project Program 3.2:

Hewer's Reinforcer for the Discrete Time State Space Model a BOEING 747 Aircraft:

```
clear all; close all; clc
```

Input Data

```
A=[0.99 0.03 -0.02 -0.32;
    0.01 0.47 4.7 0;
    0.02 -0.06 0.4 0;
    0.01 -0.04 0.72 0.99];
B=[0.01 0.99;
    -3.44 1.66;
    -0.83 0.44;
    -0.47 0.25];
x0=[1;1;1;1]; % system initial conditions
```

LQ Optimal Control Problem Solution:

```
R1=eye(4); R2=eye(2);
CO=ctrb(A,B); % forms the controllability matrix
rankCO=rank(CO) % finds the rank of the controllability matrix
% the rank is equal to n (system order) implies controllability
OM=obsv(A,R1); % forms the observability matrix
rankOM=rank(OM) % rank equal to n implies observability
[Fopt,P]=dlqr(A,B,R1,R2);
Jopt=0.5*x0'*P*x0
```

Reinforcement Learning Solution:

```
ev=eig(A);
% Since A is asymptotically stable we can use F0 = 0
F0=zeros(2,4) % pay attention to the dimension of the zero gain
% In general, to find F0 that stabilize the system, we can use the MATLAB function "place"
% that finds F0 that places the closed-loop (A-B*F0) eigenvalues in the desired locations
% lambda_desired=[-0.5;0.2;-0.1+j*0.5;-0.1-j*0.5] (note that complex conjugate eigenvalues)
% F0=place(A,B,lambda_desired); check eigenvalues of (A-B*F0); evF0=eig(A-B*F0);
```

Hewer's Reinforce

```
Iter=   6
Ji=0;
% Policy iterations of the Hewer algorithm
Pi{1} = dlyap((A-B*F0)',R1+F0'*R2*F0);
F{1} = inv(R2+B'*Pi{1}*B)*B'*Pi{1}*A;
Ji(1)=0.5*x0'*Pi{1}*x0;
Error(1) = norm((A-B*F{1})'*Pi{1}*(A-B*F{1})-Pi{1}+R1+F{1}'*R2*F{1});
for i=2:Iter
    Pi{i}=dlyap((A-B*F{i-1})',R1+F{i-1}'*R2*F{i-1});
    i;
    Ji(i)=0.5*x0'*Pi{i}*x0;
    F{i}=inv(R2+B'*Pi{i}*B)*B'*Pi{i}*A;
end
```

Figure 3.4: The first part of the considered live script

Plotting the obtained results:

```
% Performance Criterion
figure (1)
plot([1:Iter],Ji,'*')
grid on; xlabel('Iteration Number'); title('Approximate Optimal Performance')
% System running time
run_time =   50
% Iteration-wise Approximate System State Trajectories Variable
figure (2)
for i=1:Iter
    Iname = sprintf('Iteration %d', i);
    k=1:1:run_time;
    for j=1:run_time
         x(:,j)=((A-B*F{i})^k(j))*x0;
    end
    plot(k,x(1,:),'*--','DisplayName',Iname)
    hold on
end
grid on
legend('NumColumns',5,'Location','southoutside'); xlabel('Time [s]'); % ylabel('State Variable x1')
title('Iteration-wise Approximate System State Trajectories for Variable x_1')
figure (3)
for i=1:Iter
    Iname = sprintf('Iteration %d', i);
    k=1:1:run_time;
    for j=1:run_time
         x(:,j)=((A-B*F{i})^k(j))*x0;
    end
    plot(k,x(2,:),'*--','DisplayName',Iname)
    hold on
end
grid on
legend('NumColumns',5,'Location','southoutside'); xlabel('Time [s]');% ylabel('State Variable x2')
title('Iteration-wise Approximate System State Trajectories for Variable x_2')
figure (4)
for i=1:Iter
    Iname = sprintf('Iteration %d', i);
    k=1:1:run_time;
    for j=1:run_time
         x(:,j)=((A-B*F{i})^k(j))*x0;
    end
    plot(k,x(3,:),'*--','DisplayName',Iname)
    hold on
end
grid on
legend('NumColumns',5,'Location','southoutside'); xlabel('Time [s]'); % ylabel('State Variable x3')
title('Iteration-wise Approximate System State Trajectories for Variable x_3')
figure (5)
for i=1:Iter
    Iname = sprintf('Iteration %d', i);
    k=1:1:run_time;
    for j=1:run_time
         x(:,j)=((A-B*F{i})^k(j))*x0;
    end
    plot(k,x(4,:),'*--','DisplayName',Iname)
    hold on
end
grid on
legend('NumColumns',5,'Location','southoutside')
xlabel('Time [s]'); % ylabel('State Variable x4')
title('Iteration-wise Approximate System State Trajectories for Variable x_4')
```

Figure 3.5: The second part of the considered live script

```
rankCO = 4

rankOM = 4

Jopt = 14.9453

F0 = 2×4
     0     0     0     0
     0     0     0     0

Iter = 6
```

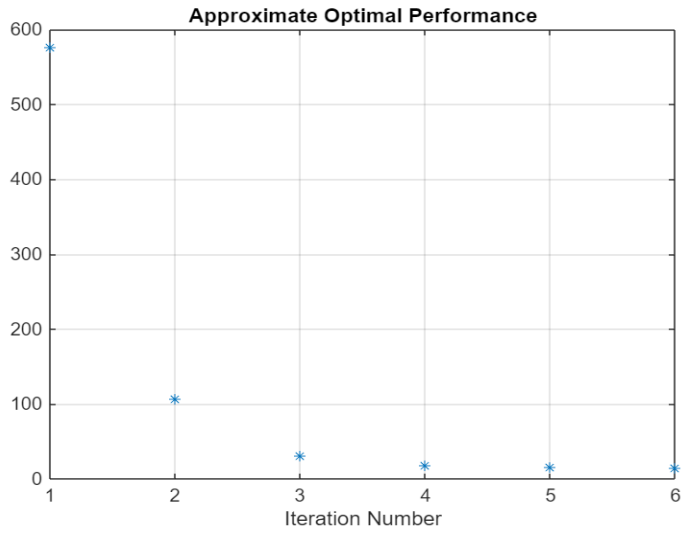
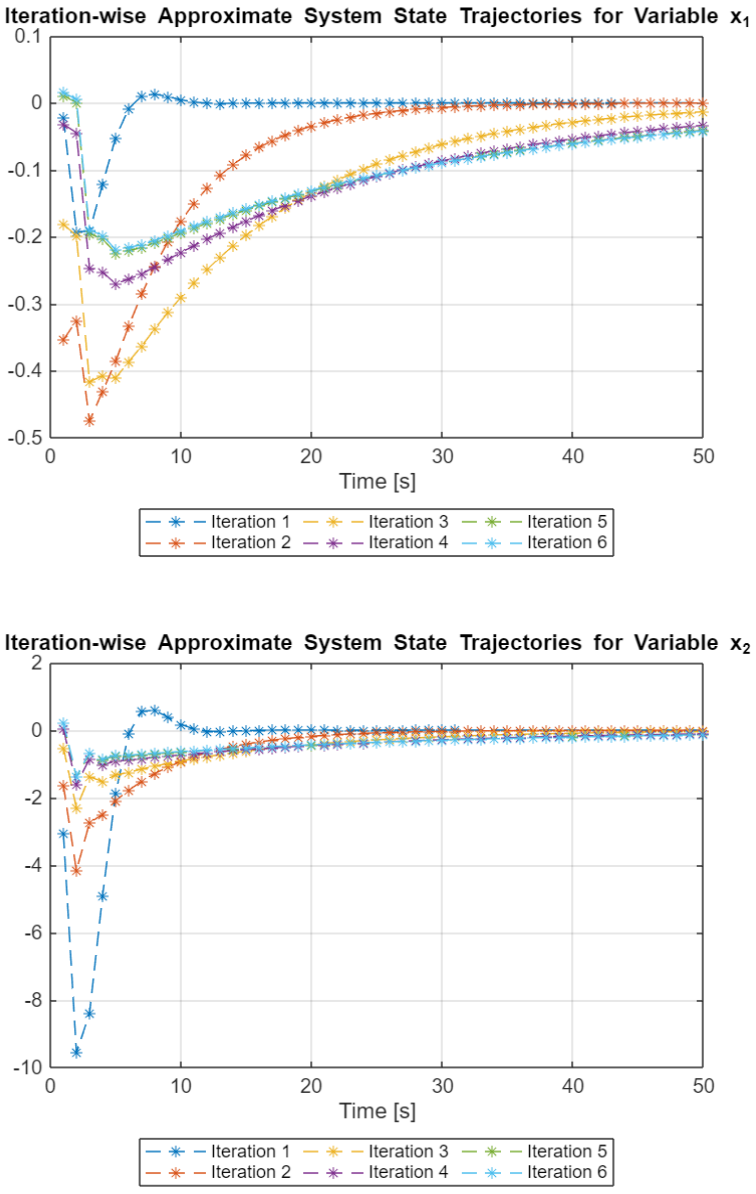


Figure 3.6: Approximate optimal performance values

The results provided on this page indicate that the studied LQ control problem satisfies controllability-observability Assumption 2.1 so that the optimal solution exists with the optimal performance being given by $J^{\text{opt}} = 14.9453$. The initial feedback gain needed for initialization of the Hewer algorithm is taken as a zero matrix of dimensions 2 x 4. The approximate values of the optimal performance criterion are presented in the given figure, indicating that they converge quickly, in five to six iterations to the optimal value. The approximate and optimal state trajectories are presented in the next four figures of this section.

Figure 3.7: Approximate and optimal state trajectories $x_1(t)$ and $x_2(t)$



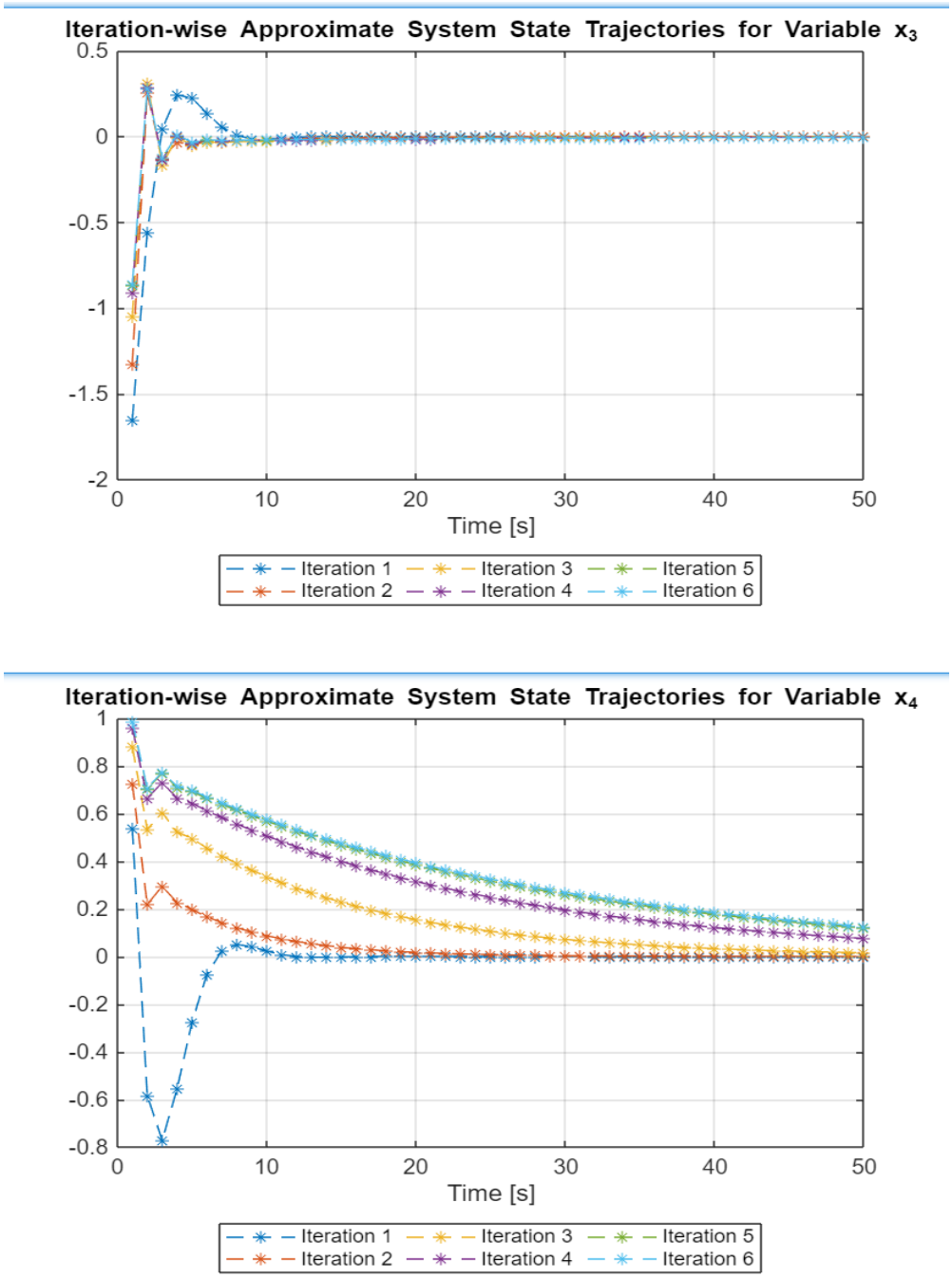


Figure 3.8: Approximate and optimal state trajectories $x_3(t)$ and $x_4(t)$

3.5.1 RL Policy Iterations Design Project for a Six Degree of Freedom Drone

A twelfth-order mathematical model for a six degrees of freedom drone was derived in several publications, see for example [24]. In this design project, the Hewer algorithm was used to implement RL policy iterations for the discrete-time LQ optimal control problem. The state variables are the roll-Θ, pitch-Φ, and yaw-Ψ angles and their angular velocities (derivatives) and x, y, z coordinates and their velocities (derivatives). The state space vector is defined by

$$X(t) = \begin{bmatrix} \Phi(t) & \dot{\Phi}(t) & \Theta(t) & \dot{\Theta}(t) & \Psi(t) & \dot{\Psi}(t) & x(t) & y(t) & z(t) & \dot{x}(t) & \dot{y}(t) & \dot{z}(t) \end{bmatrix}^T$$

The continuous-time state space model $\dot{X}(t) = AX(t) + BU(t)$ matrices are given by

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & g & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ -g & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}, \quad B = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & b_1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & b_2 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & b_3 \\ 0 & 0 & 0 & 0 \\ \frac{1}{m} & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

$$C = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}, \quad D = 0^{6\times 4}$$

where $b_1 = l / I_{xx}, b_2 = l / I_{yy}, b_3 = l / I_{zz}$. The penalty (weighted) matrices were taken in [24] as $R_1 = \text{diag}\{10,10,10,10,10,10,10,10,8,8,8,8\}$, $R_2 = I_4$. Mass of the quadcopter is $m = 0.506\,\text{kg}$. The distance from the center of the quadcopter and the DC motor is $l = 0.235\,\text{m}$. The moments of

inertia are: $I_{xx} = I_{yy} = 8.11858 \times 10^{-5}\ \text{kg m}^2$, and $I_{zz} = 6.12233 \times 10^{-5}\ \text{kg m}^2$. The gravitational constant is $g = 9.81\,\text{m/s}$.

It is important to observe that this model is both controllable and observable despite a lot of zeros in the state space matrices. The model is discretized using the MATLAB function `c2d`. Note that controllability and observability are invariant under sampling [25].

MATLAB RL-Design-Project Program 3.3:

In this demo we have run Hewer's PIs iterations algorithm for a drone model with slightly different parameters. The system and the input matrices are the same and given in the MATLAB live script. The weighted matrices are taken as identities as well as the system output matrix since all state variables are needed for feedback. The RL PIs algorithm has produced fast convergence of the discrete-time approximate state and control trajectories to the optimal ones. A selected state trajectory is plotted to demonstrate convergence.

```
clear all; close all; clc
```

Drone control Hewer vs. RL toolbox

```
% Drone 6DOF (six degrees of freedom) linearized and discretized model
g=9.81; % acceleration of gravity;
m=0.547; % [kg] quadcopter mass;
Ix=0.0033; % [kg m^2] inertia for xb
Iy=0.0033; % [kg m^2] inertia for yb
Iz=0.0058; % [kg m^2] inertia for zb

Ac=[0 0 0 0 0 0 1 0 0 0 0 0;
   0 0 0 0 0 0 0 1 0 0 0 0;
   0 0 0 0 0 0 0 0 1 0 0 0;
   0 0 0 0 0 0 0 0 0 1 0 0;
   0 0 0 0 0 0 0 0 0 0 1 0;
   0 0 0 0 0 0 0 0 0 0 0 1;
   0 0 0 0 g 0 0 0 0 0 0 0;
   0 0 0 -g 0 0 0 0 0 0 0 0;
   0 0 0 0 0 0 0 0 0 0 0 0;
   0 0 0 0 0 0 0 0 0 0 0 0;
   0 0 0 0 0 0 0 0 0 0 0 0;
   0 0 0 0 0 0 0 0 0 0 0 0];
Bc=[0 0 0 0;
   0 0 0 0;
   0 0 0 0;
```

```
    0 0 0 0;
    0 0 0 0;
    0 0 0 0;
    0 0 0 0;
    0 0 0 0;
    1/m 0 0 0;
    0 1/Ix 0 0;
    0 0 1/Iy 0;
    0 0 0 1/Iz];
n = size(Ac,1); m = size(Bc,2);
R1=eye(n); R2=eye(m);
Q = R1; R = R2;
Cc = eye(n); Dc = zeros(n,m);
```

Discretization

```
sysc = ss(Ac,Bc,Cc,Dc);
Ts = 0.01;
sysd = c2d(sysc,Ts);
A = sysd.A; B = sysd.B; C = sysd.C; D = sysd.D;
x0 = 0.5*ones(n,1);
```

Test for controllability and observability

```
CO=ctrb(A,B);
rankCO=rank(CO) % finds the rank of the controllability matrix
```

```
rankCO = 12
```

```
% the rank is equal to n (system order) implies controllability
OM=obsv(A,C);
rOM=rank(OM) % rank equal to n implies observability
```

```
rOM = 12
```

Direct solution of the optimal LQ problem

```
[Fopt,P]=dlqr(A,B,R1,R2);
Jopt=0.5*x0'*P*x0
```

```
Jopt = 279.7263
```

Reinforcement Learning Solution:

```
ev=eig(A);
% % Since A is asymptotically stable (all eigenvalues inside of unit circle
```

```
% % we can use F0 = 0 (pay attention to its dimensions F0=zeros(2,4)
% In general, to find F0 that stabilize the system,
% we can use the MATLAB function place that finds the feedback
% matrix that places the closed loo (A-B*F0) ev in the desired locations
% lambda_desired=[-0.5; 0.2; -0.1+j*0.5 -0.1-j*0.5] can use complex conjugate e
% eigenvalues F0=place(A,B,lambda_desired);
% check eigenvalues of (A-B*F0); evF0=eig(A-B*F0);
lambda_desired=0.1*[-1 -2 -3 -4 -5 -6 -7 -8 -9 0.5 8.5 7.5];
F0=place(A,B,lambda_desired);
```

Hewer's Reinforcer

$$P_i = \left(A - BF_{i-1}\right)^{\mathrm{T}} P_i \left(A - BF_{i-1}\right) + F_{i-1}^{\mathrm{T}} R F_{i-1} + Q, \qquad i = 1, 2, 3, \cdots$$

$$F_i = -\left(R + B^{\mathrm{T}} P_i B\right)^{-1} B^{\mathrm{T}} P_i A, \qquad \text{with } F_0 \text{ stabilizing } |\lambda_j(A - BF_0)| < 1$$

```
% Set-up total iteration
Iter=8
```

```
Iter = 8
```

```
Ji=0;
% Pi(:) = [];
% Policy iterations of the Hewer algorithm
Pi{1} = dlyap((A-B*F0)',R1+F0'*R2*F0);
F{1} = inv(R2+B'*Pi{1}*B)*B'*Pi{1}*A;
Ji(1)=0.5*x0'*Pi{1}*x0;
Error(1) = norm((A-B*F{1})'*Pi{1}*(A-B*F{1})-Pi{1}+R1+F{1}'*R2*F{1});
for i=2:Iter
    Pi{i}=dlyap((A-B*F{i-1})',R1+F{i-1}'*R2*F{i-1});
    i;
    Ji(i)=0.5*x0'*Pi{i}*x0;
    F{i}=inv(R2+B'*Pi{i}*B)*B'*Pi{i}*A;
    Error(i) = norm((A-B*F{i})'*Pi{i}*(A-B*F{i})-Pi{i}+R1+F{i}'*R2*F{i});
% Calling Simulink
    model=sim(Drone12);
    time_itr(1:length(model.t),i) = model.t;
    X(i,1:length(model.t),:) = model.x(:,:); % i-th iter, step, x(j)
end
F{i} = Fopt;
model=sim(Drone12);
```

```
% Plotting approximate and optimal state trajectories
% time_itr(1:length(model.t),i) = model.t;
X_opt(1:length(model.t),:) = model.x(:,:); % i-th iter, step, x(j)
```

Plotting the obtained results:

```
markers = {'o', '+', '*', '.', 'x', 's', 'd', '^', 'v', '>', '<', 'p', 'h'};
lineStyles = {'-', '--', ':', '-.'};

figure (1)
Iname_1 = sprintf('Iteration opt');
plot(time_itr(:,2),X_opt(:,1),'Marker','x' ,...
         'linestyle',':','DisplayName',Iname_1)
 for i=1:Iter
     hold on
     Iname_1 = sprintf('Iteration %d', i);
     plot(time_itr(:,i),X(i,:,1),'Marker',markers{mod(i,Iter+1)},...
         'linestyle',lineStyles{max(mod(i,4),1)},'DisplayName',Iname_1)
 end
grid on
set(gca,'GridLineStyle','--')
xlabel('Time [s]');title('State variable x_1')
legend;
```

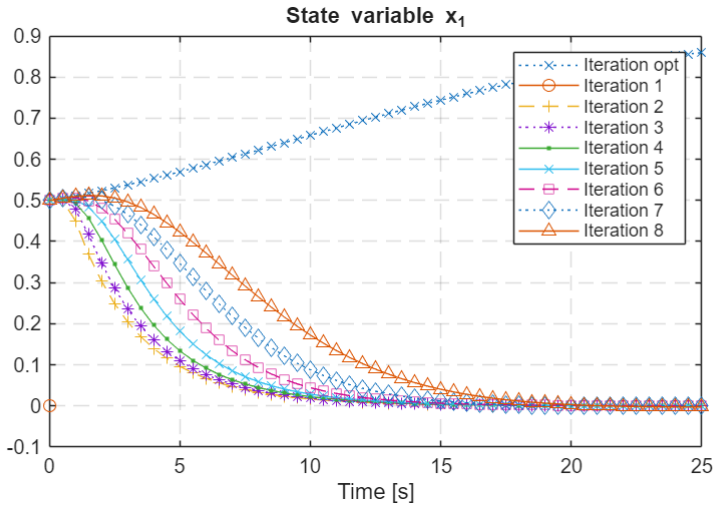


Figure 3.9: Approximate and optimal state trajectories $x_1(t)$

3.6 References

[1] Bellman, R., *Dynamic Programming*, Princeton University Press, Princeton, NJ, 1957, republished by Dover Publications, 2003.

[2] Bellman, R., *Adaptive Control Processes: A Guider Tour*, Princeton University Press, NJ, 1961.

[3] Hewer, G., "An iterative technique for the computation of the steady state gains for the discrete optimal regulator," *IEEE Transactions on Automatic Control*, Vol. AC-16, 382-384, 1971.

[4] Hewer, G., "Analysis of a discrete matrix Riccati equation of linear control and Kalman filter, *Journal of Mathematical Analysis and Applications*, Vol. 42, 226-236, 1973.

[5] Kleinman, D., "On an iterative technique for Riccati equations computations," *IEEE Transactions on Automatic Control*," Vol. 13, 114-115, 1968.

[6] Bellman, R. and S. Dreyfus, *Applied Dynamic Programming*, Princeton University Press, Princeton, NJ 1962.

[7] Bellman, R. and R. Kalaba, *Quasilinearization and Nonlinear Boundary-Value Problem*, Elsevier, New York, 1965.

[8] Larson, R., "A survey of dynamic programming computational procedures," *IEEE Transactions on Automatic Control*, Vol. AC-12, 767-774, 1967.

[9] Larson, R., *State Increment Dynamic Programming*, Elsevier, New York, 1968.

[10] Larson, R. and J. Casti, *Principles of Dynamic Programming: Part I*. Marcel Dekker, New York, 1978.

[11] Murray, J., C. Cox, G. Lendaris, and R. Saeks, "Adaptive dynamic programming," *IEEE Transactions on Systems, Man, and Cybernetics- Part C: Applications and Reviews*, Vol. 32, 140-153, 2002.

[12] Tang, G-Y. and H-H. Wang, "Successive approximation of optimal control for nonlinear discrete-time systems," *International Journal of Systems Science*, Vol. 36, 153-11, 2005.

[13] Al-Tamimi, A., F. Lewis, and M. Abu-Khalif, "Discrete-time nonlinear HJB solution using approximate dynamic programming: Convergence proofs," *IEEE Transactions on Systems, Man, and Cybernetics-Part B: Cybernetics*, Vol. 38, 943-949, 2008.

[14] Lewis, F. and D. Vrabie, "Reinforcement learning and feedback control," *IEEE Circuits and Systems Magazine*, 32-50, Third Quarter, 2009.

[15] Lewis, F., D. Vrabie, and K. Vamvoudakis, "Reinforcement learning and feedback control," *IEEE Control Systems Magazine*, 76-105, Dec. 2012.

[16] Lewis, F. and D. Liu, Eds., *Reinforcement Learning and Approximate Dynamic Programming for Feedback Control*, Hoboken, New Jersey, 2013.

[17] Liu, D. and Q. Wei, "Finite-approximation-error-based optimal control approach for discrete-time nonlinear systems," *IEEE Transactions on Cybernetics*, Vol. 43, 779-789, 2013.

[18] Liu, D. and Q. Wei, "Policy iteration dynamic programming algorithm for discrete-time nonlinear systems," *IEEE Transactions on Neural Networks and Learning Systems*," Vol. 25, 621-634, 2014.

[19] Wei, Q., F-Y. Wang, D. Liu, and X. Yang, "Finite-approximation-error-based discrete-time iterative adaptive dynamic programming," *IEEE Transactions on Cybernetics*, Vol. 44, 2820-2833, 2014.

[20] Bertsekas, D., *Dynamic Programming and Optimal Control: Approximate Dynamic Programming*, Athina Scientific, Belmont, MA, 2012.

[21] Bertsekas, D., *Reinforcement Learning and Optimal Control*, Athina Scientific, Belmont, MA, 2019.

[22] Howard, R., *Dynamic Programming and Markov Processes*, MIT Press, Cambridge, MA, 1960.

[23] Goel, G. and B. Hassibi, "Competitive Control," *IEEE Transactions on Automatic Control*, Vol. 68, 3162-5173, 2023.

[24] Kurak, S. and M. Hodzic, "Control and estimation of a quadcopter dynamical model," *Periodicals of Engineering and Natural Sciences*, Vol. 6, 63-75, 2018.

[25] Chen, T-C., *Linear Systems: Theory and Design*, Oxford University Press, Oxford, U.K., 2013.